

10.1 Lists and Sets in Java

In this section, we will learn about how to use Java's built-in `List` and `Set` data structures as well as build our own `ArraySet`.

Getting Started

```
{% embed url="https://youtu.be/DWr8YNXPH6k" %}
```

In this course, we've already built two kinds of lists: `AList` and `SLList`. We also built an interface `List61B` to enforce specific list methods `AList` and `SLList` had to implement. You can find the code at the following links:

- [List61B](#)
- [AList](#)
- [SLList](#)

This is how we might use `List61B` type:

```
List61B<Integer> L = new AList<>();
L.addLast(5);
L.addLast(10);
L.addLast(15);
L.print();
```

Lists in Real Java Code

We built a list from scratch, but Java provides a built-in `List` interface and several implementations, e.g. `ArrayList`. Remember, since `List` is an interface we can't instantiate it! We must instantiate one of its implementations.

To access this, we can use the full name ('canonical name') of classes/interfaces:

```
java.util.List<Integer> L = new java.util.ArrayList<>();
```

However, this is a bit verbose. Instead, we can import java libraries:

```
import java.util.List;
import java.util.ArrayList;

public class Example {
    public static void main(String[] args) {
        List<Integer> L = new ArrayList<>();
        L.add(5);
        L.add(10);
        System.out.println(L);
    }
}
```

Sets

Sets are a collection of unique elements - you can only have one copy of each element. Unlike Lists, there is also no sense of order: you can't index into a set, nor can you control where each element is inserted into the set.

Java Sets

Java has the `Set` interface along with implementations, e.g. `HashSet`. Remember to import them if you don't want to use the full name!

```
import java.util.Set;
import java.util.HashSet;
```

Example use:

```
Set<String> s = new HashSet<>();
s.add("Tokyo");
s.add("Lagos");
System.out.println(s.contains("Tokyo")); // true
```

Python Equivalent

In python, we simply call `set()`. To check for contains we don't use a method but the keyword `in`. Here's an example:

```
s = set()
s.add("Tokyo")
s.add("Lagos")
print("Tokyo" in s) // True
```

DIY: ArraySet

Our goal is to make our own set, `ArraySet`, with the following methods:

- `add(value)`: add the value to the set if not already present
- `contains(value)`: check to see if `ArraySet` contains the key
- `size()`: return number of values

If you would like to try it yourself, find 'Do It Yourself' `ArraySet` starter code [here](#). In the lecture clip below, Professor Hug goes develops the solution:

```
{% embed url="https://youtu.be/gX9KFdZBg-k" %}
```

10.2 Exceptions

In this section, we will learn how to throw exceptions to effectively handle errors that may arise in our code.

```
{% embed url="https://youtu.be/r5hp67RfWaY" %}
```

Our `ArraySet` implementation from the previous section has a small error. When we add `null` to our `ArraySet`, we get a `NullPointerException`.

The problem lies in the `contains` method where we check `items[i].equals(x)`. If the value at `items[i]` is `null`, then we are calling `null.equals(x)` -> `NullPointerException`.

Exceptions cause normal flow of control to stop. We can in fact choose to throw our own exceptions. In python you may have seen this with the `raise` keyword. In Java, Exceptions are objects and we throw exceptions using the following format:

```
throw new ExceptionObject(parameter1, ...)
```

Let's throw an exception when a user tries to add `null` to our `ArraySet`. We'll throw an `IllegalArgumentException` which takes in one parameter (a `String` message).

Our updated `add` method:

```
/* Associates the specified value with the specified key in this map.
 * Throws an IllegalArgumentException if the key is null. */
public void add(T x) {
    if (x == null) {
        throw new IllegalArgumentException("can't add null");
    }
    if (contains(x)) {
        return;
    }
    items[size] = x;
    size += 1;
}
```

We get an Exception either way - why does this better?

1. We have control of our code: we consciously decide at what point to stop the flow of our program
2. More useful Exception type and helpful error message for those using our code

However, it would be better if the program doesn't crash at all. There are different things we could do in this case. Here are some below:

Approach 1: Don't add `null` to the array if it is passed into `add`

Approach 2: Change the `contains` method to account for the case if `items[i] == null`.

Whatever you decide, it is important that users know what to expect. That is why documentation (such as comments about your methods) is very important.

10.3 Iteration

```
{% embed url="https://www.youtube.com/watch?v=Gv6LjusNBU0" %}
```

We can use a clean enhanced for loop with Java's `HashSet`

```
Set<String> s = new HashSet<>();
s.add("Tokyo");
s.add("Lagos");
for (String city : s) {
    System.out.println(city);
}
```

However, if we try to do the same with our `ArraySet`, we get an error. How can we enable this functionality?

Enhanced For Loop

Let's first understand what is happening when we use an enhanced for loop. We can "translate" an enhanced for loop into an ugly, manual approach.

```
Set<String> s = new HashSet<>();
...
for (String city : s) {
    ...
}
```

The above code translates to:

```
Set<String> s = new HashSet<>();
...
Iterator<String> seer = s.iterator();
while (seer.hasNext()) {
    String city = seer.next();
    ...
}
```

Let's strip away the magic so we can build our own classes that support this.

The key here is an object called an *iterator*.

For our example, in `List.java` we might define an `iterator()` method that returns an iterator object.

```
public Iterator<E> iterator();
```

Now, we can use that object to loop through all the entries in our list:

```
List<Integer> friends = new ArrayList<Integer>();
...
Iterator<Integer> seer = friends.iterator();

while (seer.hasNext()) {
    System.out.println(seer.next());
}
```

This code behaves identically to the `foreach` loop version above.

There are three key methods in our iterator approach:

First, we get a new iterator object with `Iterator<Integer> seer = friends.iterator();`

Next, we loop through the list with our while loop. We check that there are still items left with `seer.hasNext()`, which will return true if there are unseen items remaining, and false if all items have been processed.

Last, `seer.next()` does two things at once. It returns the next element of the list, and here we print it out. It also advances the iterator by one item. In this way, the iterator will only inspect each item once.

Implementing Iterators

In this section, we are going to talk about how to build a class to support iteration.

Let's start by thinking about what the compiler needs to know in order to successfully compile the following iterator example:

```
List<Integer> friends = new ArrayList<Integer>();
Iterator<Integer> seer = friends.iterator();

while(seer.hasNext()) {
    System.out.println(seer.next());
}
```

We can look at the static types of each object that calls a relevant method. `friends` is a `List`, on which `iterator()` is called, so we must ask:

- Does the `List` interface have an `iterator()` method?

`seer` is an `Iterator`, on which `hasNext()` and `next()` are called, so we must ask:

- Does the `Iterator` interface have `next/hasNext()` methods?

So how do we implement these requirements?

The `List` interface extends the `Iterable` interface, inheriting the abstract `iterator()` method. (Actually, `List` extends `Collection` which extends `Iterable`, but it's easier to codethink of this way to start.)

```
public interface Iterable<T> {
    Iterator<T> iterator();
}

public interface List<T> extends Iterable<T>{
    ...
}
```

Next, the compiler checks that `Iterators` have `hasNext()` and `next()`. The `Iterator` interface specifies these abstract methods explicitly:

```
public interface Iterator<T> {
    boolean hasNext();
    T next();
}
```

What if someone calls `next` when `hasNext` returns `false`?

This behavior is undefined. However, a common convention is to throw a `NoSuchElementException`. See [Discussion 5](https://sp19.datastructur.es/materials/discussion/disc05sol.pdf) for examples.

Will `hasNext` always be called before `next`?

Not necessarily. This is sometimes the case when someone using the iterator knows exactly how many elements are in the sequence. Thus, we can't rely on the user calling `hasNext` before `next`. However, you

Specific classes will implement their own iteration behaviors for the interface methods. Let's look at an example. (Note: if you want to build this up from the start, follow along with the live coding in the video.)

We are going to add iteration through keys to our `ArraySet` class. First, we write a new class called `ArraySetIterator`, nested inside of `ArraySet`:

```
private class ArraySetIterator implements Iterator<T> {
    private int wizPos;

    public ArraySetIterator() {
        wizPos = 0;
    }

    public boolean hasNext() {
        return wizPos < size;
    }

    public T next() {
        T returnItem = items[wizPos];
        wizPos += 1;
        return returnItem;
    }
}
```

This `ArraySetIterator` implements `Iterator<T>`, which means it implements a `hasNext()` method, and a `next()` method, using a `wizPos` position as an index to keep track of its position in the array. For a different data structure, we might implement these two methods differently.

Thought Exercise: How would you design `hasNext()` and `next()` for a linked list?

Now that we have the appropriate methods, we could use a `ArraySetIterator` to iterate through an `ArraySet`:

```
ArraySet<Integer> aset = new ArraySet<>();
aset.add(5);
aset.add(23);
aset.add(42);

Iterator<Integer> iter = aset.iterator();

while(iter.hasNext()) {
    System.out.println(iter.next());
}
```

We still want to be able to support the enhanced for loop, though, to make our calls cleaner. So, we need to make `ArraySet` implement the `Iterable` interface. The essential method of the `Iterable` interface is `iterator()`, which returns an `Iterator` object for that class. All we have to do is return an instance of our `ArraySetIterator` that we just wrote!

```
public Iterator<T> iterator() {
    return new ArraySetIterator();
}
```

Now we can use enhanced for loops with our `ArraySet`!

```
ArraySet<Integer> aset = new ArraySet<>();
...
for (int i : aset) {
    System.out.println(i);
}
```

Here we've seen **Iterable**, the interface that makes a class able to be iterated on, and requires the method `iterator()`, which returns an `Iterator` object. And we've seen **Iterator**, the interface that defines the object with methods to actually do that iteration. You can think of an `Iterator` as a machine that you put onto an iterable that facilitates the iteration. Any iterable is the object on which the iterator is performing.

With these two components, you can make fancy for loops for your classes!

`ArraySet` code with iteration support is below:

```
import java.util.Iterator;

public class ArraySet<T> implements Iterable<T> {
    private T[] items;
    private int size; // the next item to be added will be at position size

    public ArraySet() {
        items = (T[]) new Object[100];
        size = 0;
    }

    /* Returns true if this map contains a mapping for the specified key.
    */
    public boolean contains(T x) {
        for (int i = 0; i < size; i += 1) {
            if (items[i].equals(x)) {
                return true;
            }
        }
        return false;
    }

    /* Associates the specified value with the specified key in this map.
    Throws an IllegalArgumentException if the key is null. */
    public void add(T x) {
        if (x == null) {
            throw new IllegalArgumentException("can't add null");
        }
        if (contains(x)) {
            return;
        }
    }
}
```

```

        items[size] = x;
        size += 1;
    }

    /* Returns the number of key-value mappings in this map. */
    public int size() {
        return size;
    }

    /** returns an iterator (a.k.a. seer) into ME */
    public Iterator<T> iterator() {
        return new ArraySetIterator();
    }

    private class ArraySetIterator implements Iterator<T> {
        private int wizPos;

        public ArraySetIterator() {
            wizPos = 0;
        }

        public boolean hasNext() {
            return wizPos < size;
        }

        public T next() {
            T returnItem = items[wizPos];
            wizPos += 1;
            return returnItem;
        }
    }

    public static void main(String[] args) {
        ArraySet<Integer> aset = new ArraySet<>();
        aset.add(5);
        aset.add(23);
        aset.add(42);

        //iteration
        for (int i : aset) {
            System.out.println(i);
        }
    }
}

```

12.4 Object Methods

All classes inherit from the overarching Object class. The methods that are inherited are as follows:

- String toString()
- boolean equals(Object obj)
- Class <?> getClass()
- int hashCode()
- protected Object clone()
- protected void finalize()
- void notify()
- void notifyAll()
- void wait()
- void wait(long timeout)
- void wait(long timeout, int nanos)

We are going to focus on the first two in this chapter. We will take advantage of inheritance to override these two methods in our classes to behave in the ways we want them to.

toString()

```
{% embed url="https://www.youtube.com/watch?v=AKnMv0ootkg" %}
```

The `toString()` method provides a string representation of an object. The `System.out.println()` function implicitly calls this method on whatever object is passed to it and prints the string returned. When you run `System.out.println(dog)`, it's actually doing this:

```
String s = dog.toString()
System.out.println(s)
```

The default `Object` class' `toString()` method prints the location of the object in memory. This is a hexadecimal string. Classes like `ArrayList` and java arrays have their own overridden versions of the `toString()` method. This is why, when you were working with and writing tests for `ArrayList`, errors would always return the list in a nice format like this (1, 2, 3, 4) instead of returning the memory location.

For classes that we've written by ourselves like `ArrayDeque`, `LinkedListDeque`, etc, we need to provide our own `toString()` method if we want to be able to see the objects printed in a readable format.

Let's try to write this method for an `ArraySet` class. Read the `ArraySet` class below and make sure you understand what the various methods do. Feel free to plug the code into java visualizer to get a better understanding!

```
import java.util.Iterator;

public class ArraySet<T> implements Iterable<T> {
    private T[] items;
    private int size; // the next item to be added will be at position size

    public ArraySet() {
        items = (T[]) new Object[100];
        size = 0;
    }

    /* Returns true if this map contains a mapping for the specified key.
    */
    public boolean contains(T x) {
        for (int i = 0; i < size; i += 1) {
            if (items[i].equals(x)) {
                return true;
            }
        }
        return false;
    }

    /* Associates the specified value with the specified key in this map.
    Throws an IllegalArgumentException if the key is null. */
    public void add(T x) {
        if (x == null) {
            throw new IllegalArgumentException("can't add null");
        }
        if (contains(x)) {
            return;
        }
        items[size] = x;
        size += 1;
    }

    /* Returns the number of key-value mappings in this map. */
    public int size() {
        return size;
    }

    /** returns an iterator (a.k.a. seer) into ME */
    public Iterator<T> iterator() {
        return new ArraySetIterator();
    }

    private class ArraySetIterator implements Iterator<T> {
        private int wizPos;

        public ArraySetIterator() {
            wizPos = 0;
        }

        public boolean hasNext() {
            return wizPos < size;
        }

        public T next() {
            T returnItem = items[wizPos];
            wizPos += 1;
        }
    }
}
```

```

        return returnItem;
    }
}

@Override
public String toString() {
    /* hmmm */
}

@Override
public boolean equals(Object other) {
    /* hmmm */
}

public static void main(String[] args) {
    ArraySet<Integer> aset = new ArraySet<>();
    aset.add(5);
    aset.add(23);
    aset.add(42);

    //iteration
    for (int i : aset) {
        System.out.println(i);
    }

    //toString
    System.out.println(aset);

    //equals
    ArraySet<Integer> aset2 = new ArraySet<>();
    aset2.add(5);
    aset2.add(23);
    aset2.add(42);

    System.out.println(aset.equals(aset2));
    System.out.println(aset.equals(null));
    System.out.println(aset.equals("fish"));
    System.out.println(aset.equals(aset));
}

```

You can find the [solutions here \(ArraySet.java\)](#)

Exercise 6.4.1: Write the `toString()` method so that when we print an `ArraySet`, it prints the elements separated by commas inside of curly braces. i.e `{1, 2, 3, 4}`. Remember, the `toString()` method should return a string.

Solution

```

public String toString() {
    String returnString = "{";
    for (int i = 0; i < size; i += 1) {
        returnString += keys[i];
        returnString += ", ";
    }
    returnString += "}";
    return returnString;
}

```

This solution, although seemingly simple and elegant, is actually very naive. This is because when you use string concatenation in Java like so: `returnString += keys[i];` you are actually not just appending to `returnString`, you are creating an entirely new string. This is incredibly inefficient because creating a new string object takes time too! Specifically, linear in the length of the string (we'll discuss why in a later chapter!).

Bonus Question: Let's say concatenating one character to a string takes 1 second. If we have an `ArraySet` of size 5: `{1, 2, 3, 4, 5}`, how long would it take to run the `toString()` method?

Answer: We set `returnString` to the left bracket which takes one second because this involves adding `{` to the empty string `""`. Adding the first element will involve creating an entirely new string, adding `}` and 1 which would take 2 seconds. Adding the second element takes 3 seconds because we need to add `{, 1, 2`. This process continues, so for the entire array set the total time is $1 + 2 + 3 + 4 + 5 + 6 + 7$.

To remedy this, Java has a special class called `StringBuilder`. It creates a string object that is mutable, so you can continue appending to the same string object instead of creating a new one each time.

Exercise 6.4.2: Rewrite the `toString()` method using `StringBuilder`.

Solution


```

public String toString() {
    StringBuilder returnSB = new StringBuilder("{}");
    for (int i = 0; i < size - 1; i += 1) {
        returnSB.append(items[i].toString());
        returnSB.append(", ");
    }
    returnSB.append(items[size - 1]);
    returnSB.append("}");
    return returnSB.toString();
}

```

Now you've successfully overridden the `toString()` method! Try printing the `ArraySet` to see the fruits of your work.

Next we will override another important object method: `equals()`

equals()

{% embed url="https://www.youtube.com/watch?v=qHuS1o97nfQ" %}

`equals()` and `==` have different behaviors in Java. `==` Checks if two objects are actually the same object in memory. Remember, pass-by-value! `==` checks if two boxes hold the same thing. For primitives, this means checking if the values are equal. For objects, this means checking if the address/pointer is equal.

Say we have this `Doge` class:

```

public class Doge {

    public int age;
    public String name;

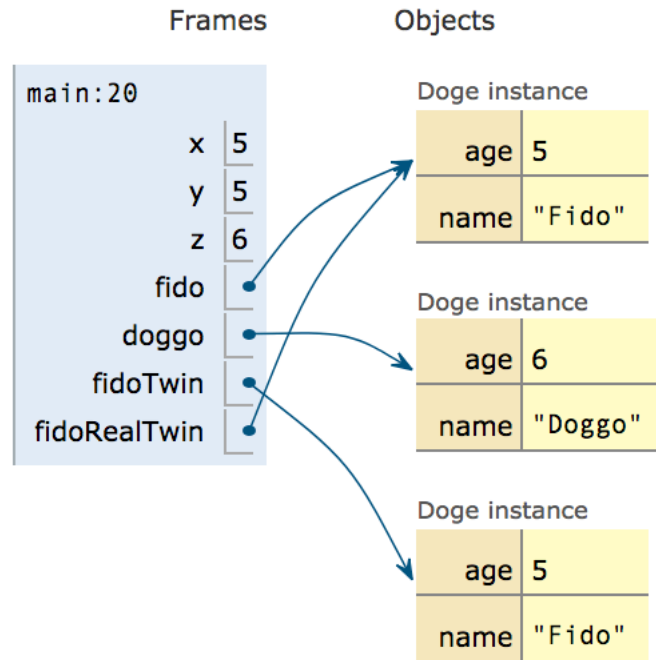
    public Doge(int age, String name){
        this.age = age;
        this.name = name;
    }
    public static void main(String[] args) {

        int x = 5;
        int y = 5;
        int z = 6;

        Doge fido = new Doge(5, "Fido");
        Doge doggo = new Doge(6, "Doggo");
        Doge fidoTwin = new Doge(5, "Fido");
        Doge fidoRealTwin = fido;
    }
}

```

If we plug this code into the java visualizer, we will see the box in pointer diagram shown below.



Exercise 6.4.3: What would java return if we ran the following?

- `x == y`
- `x == z`
- `fido == doggo`
- `fido == fidoTwin`
- `fido == fidoRealTwin`

Answers

- True
- False
- False
- False
- True

`fido` and `fidoTwin` are not considered `==` because they point to different objects. However, this is quite silly since all their attributes are the same. You can see how `==` can cause some problems in Java testing. When we write tests for our ArrayList and want to check if expected is the same as what is returned by our function, we create expected as a new arraylist. If we used `==` in our test, it would always return false. This is what `equals(Object o)` is for.

`equals(Object o)`

`equals(Object o)` is a method in the Object class that, by default, acts like `==` in that it checks if the memory address of the this is the same as o. However, we can override it to define equality in whichever way we wish! For example, for two ArrayLists to be considered equal, they just need to have the same elements in the same order.

Exercise 6.4.4: Let's write an equals method for the ArraySet class. Remember, a set is an unordered collection of unique elements. So, for two sets to be considered equal, you just need to check if they have the same elements.

Solution

```

public boolean equals(Object other) {
    if (this == other) {
        return true;
    }
    if (other == null) {
        return false;
    }
    if (other.getClass() != this.getClass()) {
        return false;
    }
    ArraySet<T> o = (ArraySet<T>) other;
    if (o.size() != this.size()) {
        return false;
    }
    for (T item : this) {
        if (!o.contains(item)) {
            return false;
        }
    }
    return true;
}

```

We added a few checks in the beginning of the method to make sure our `.equals()` can handle nulls and objects of a different class. We also optimized the function by returning true right away if the `==` methods returns true. This way, we avoid the extra work of iterating through the set.

Rules for Equals in Java: Overriding a `.equals()` method may sometimes be trickier than it seems. A couple of rules to adhere to while implementing your `.equals()` method are as follows:

1.) equals must be an equivalence relation

- **reflexive:** `x.equals(x)` is true
- **symmetric:** `x.equals(y)` if and only if `y.equals(x)`
- **transitive:** `x.equals(y)` and `y.equals(z)` implies `x.equals(z)`

2.) It must take an `Object` argument, in order to override the original `.equals()` method

3.) It must be consistent if `x.equals(y)`, then as long as `x` and `y` remain unchanged: `x` must continue to equal `y`

4.) It is never true for null `x.equals(null)` must be false

Bonus video

Create an even better `toString` method and `ArraySet.of`:

```
{% embed url="https://www.youtube.com/watch?v=tjLpeVD0KWc" %}
```

Link to the [bonus code](#)

10.5 Chapter Summary

You can find the code from this lecture [here](#).

Exceptions

Most likely you have encountered an exception in your code such as a `NullPointerException` or an `IndexOutOfBoundsException`. Now we will learn about how we can “throw” exceptions ourselves. Here is an example of an exception that we throw:

```
throw new RuntimeException("For no reason.");
```

This is useful to ensure reasonable functioning of our code, even when facing unexpected behavior.

Iteration

Difference between Iterators and Iterables

These two words are very closely related, but have two different meanings that are often easy to confuse. The first thing to know is that these are both Java interfaces, with different methods that need to be implemented. Here is a simplified interface for `Iterator`:

```
public interface Iterator<T> {
    boolean hasNext();
    T next();
}
```

Here is a simplified interface for `Iterable`:

```
public interface Iterable<T> {
    Iterator<T> iterator();
}
```

Notice that in order for an object (for example an `ArrayList` or `LinkedList`) to be *iterable*, it must include a method that returns an *iterator*. The iterator is the object that actively steps through an iterable object. Keep this relationship and distinction in mind as you work with these two interfaces.

Object Methods

toString

The `toString()` method returns a string representation of objects. For example, `System.out.println(someObject)` calls the `toString()` method of `someObject`, and prints to console whatever string it returns.

This is most helpful when we are debugging, as it allows us to much more easily understand the current state of our Objects.

== vs .equals

We have two concepts of equality in Java- “==” and the “.equals()” method. The key difference is that when using `==`, we are checking if two objects have the same address in memory (that they point to the same instance or object). On the other hand, `.equals()` is a method that can be overridden by a class and can be used to define some custom way of determining equality. This permits the class to utilize the additional knowledge it has about itself to more accurately answer questions of equality.

For example, say we wanted to check if two stones are equal:

```
public class Stone{
    int weight;
    public Stone(int weight){
        this.weight = weight;
    }
}
Stone s = new Stone(100);
Stone r = new Stone(100);
```

If we want to consider `s` and `r` equal because they have the same weight. If we do check equality using `==`, these Stones would not be considered equal because they do not have the same memory address.

On the other hand, if you override the `equals` method of `Stone` as follows

```
public boolean equals(Object o){
    return this.weight == ((Stone) o).weight
}
```

We would have that the stones would be considered equal because they have the same weight.

10.6 Exercises

Factual

1. What methods are required for a class that is `Iterable`?
2. Which of the following is true about the `java.util.Set` and the `java.util.List` interfaces?
 - ☐ If we add `String[][]` objects to a `Set` and a `List`, the size of the set will always be less than or equal to the size of the list.
 - ☐ The `java.util.ArrayList` class is an implementation of the `java.util.List` interface.
 - ☐ The `Set` and `List` interfaces extend the `Iterator` interface.
 - ☐ The `Set` and `List` interfaces extend the `Iterable` interface.

3. Suppose we have a class that implements `Iterator`. What methods must it override in order to compile?

- Problem 1
- Problem 2
- Problem 3

Conceptual

1. Why do we want to override the `.equals` method?

- Problem 1

Metacognitive

1. In lecture, you built the `ArraySetIterator` class. Modify the lecture class to take in a `Comparator<T>` and an item of generic type `T` called `ref` in the constructor. This new iterator should only return items greater than `ref`. For reference, the code for `ArraySetIterator` is included below.

```
private class ArraySetIterator implements Iterator<T> {
    private int pos;

    public ArraySetIterator() {
        pos = 0;
    }

    public boolean hasNext() {
        return pos < size;
    }

    public T next() {
        T returnItem = items[pos];
        pos += 1;
        return returnItem;
    }
}
```

2. Problem 7 from the Spring 2018 Midterm 2

- Problem 1
- Problem 2

10.4 Object Methods

All classes inherit from the overarching `Object` class. The methods that are inherited are as follows:

- `String toString()`
- `boolean equals(Object obj)`
- `Class <?> getClass()`
- `int hashCode()`
- `protected Object clone()`
- `protected void finalize()`
- `void notify()`
- `void notifyAll()`
- `void wait()`
- `void wait(long timeout)`
- `void wait(long timeout, int nanos)`

We are going to focus on the first two in this chapter. We will take advantage of inheritance to override these two methods in our classes to behave in the ways we want them to.

toString()

```
{% embed url="https://www.youtube.com/watch?v=AKnMv0ootkg" %}
```

The `toString()` method provides a string representation of an object. The `System.out.println()` function implicitly calls this method on whatever object is passed to it and prints the string returned. When you run `System.out.println(dog)`, it's actually doing this:

```
String s = dog.toString()
System.out.println(s)
```

The default `Object` class' `toString()` method prints the location of the object in memory. This is a hexadecimal string. Classes like `ArrayList` and java arrays have their own overridden versions of the `toString()` method. This is why, when you were working with and writing tests for `ArrayList`, errors would always return the list in a nice format like this (1, 2, 3, 4) instead of returning the memory location.

For classes that we've written by ourselves like `ArrayDeque`, `LinkedListDeque`, etc, we need to provide our own `toString()` method if we want to be able to see the objects printed in a readable format.

Let's try to write this method for an `ArraySet` class. Read the `ArraySet` class below and make sure you understand what the various methods do. Feel free to plug the code into java visualizer to get a better understanding!

```
import java.util.Iterator;

public class ArraySet<T> implements Iterable<T> {
    private T[] items;
    private int size; // the next item to be added will be at position size

    public ArraySet() {
        items = (T[]) new Object[100];
        size = 0;
    }

    /* Returns true if this map contains a mapping for the specified key.
    */
    public boolean contains(T x) {
        for (int i = 0; i < size; i += 1) {
            if (items[i].equals(x)) {
                return true;
            }
        }
        return false;
    }

    /* Associates the specified value with the specified key in this map.
    Throws an IllegalArgumentException if the key is null. */
    public void add(T x) {
        if (x == null) {
            throw new IllegalArgumentException("can't add null");
        }
        if (contains(x)) {
            return;
        }
        items[size] = x;
        size += 1;
    }

    /* Returns the number of key-value mappings in this map. */
    public int size() {
        return size;
    }

    /** returns an iterator (a.k.a. seer) into ME */
    public Iterator<T> iterator() {
        return new ArraySetIterator();
    }

    private class ArraySetIterator implements Iterator<T> {
        private int wizPos;

        public ArraySetIterator() {
            wizPos = 0;
        }

        public boolean hasNext() {
            return wizPos < size;
        }

        public T next() {
            T returnItem = items[wizPos];
            wizPos += 1;
        }
    }
}
```

```

        return returnItem;
    }
}

@Override
public String toString() {
    /* hmmm */
}

@Override
public boolean equals(Object other) {
    /* hmmm */
}

public static void main(String[] args) {
    ArraySet<Integer> aset = new ArraySet<>();
    aset.add(5);
    aset.add(23);
    aset.add(42);

    //iteration
    for (int i : aset) {
        System.out.println(i);
    }

    //toString
    System.out.println(aset);

    //equals
    ArraySet<Integer> aset2 = new ArraySet<>();
    aset2.add(5);
    aset2.add(23);
    aset2.add(42);

    System.out.println(aset.equals(aset2));
    System.out.println(aset.equals(null));
    System.out.println(aset.equals("fish"));
    System.out.println(aset.equals(aset));
}

```

You can find the [solutions here \(ArraySet.java\)](#)

Exercise 6.4.1: Write the `toString()` method so that when we print an `ArraySet`, it prints the elements separated by commas inside of curly braces. i.e `{1, 2, 3, 4}`. Remember, the `toString()` method should return a string.

Solution

```

public String toString() {
    String returnString = "{";
    for (int i = 0; i < size; i += 1) {
        returnString += keys[i];
        returnString += ", ";
    }
    returnString += "}";
    return returnString;
}

```

This solution, although seemingly simple and elegant, is actually very naive. This is because when you use string concatenation in Java like so: `returnString += keys[i];` you are actually not just appending to `returnString`, you are creating an entirely new string. This is incredibly inefficient because creating a new string object takes time too! Specifically, linear in the length of the string (we'll discuss why in a later chapter!).

Bonus Question: Let's say concatenating one character to a string takes 1 second. If we have an `ArraySet` of size 5: `{1, 2, 3, 4, 5}`, how long would it take to run the `toString()` method?

Answer: We set `returnString` to the left bracket which takes one second because this involves adding `{` to the empty string `""`. Adding the first element will involve creating an entirely new string, adding `}` and 1 which would take 2 seconds. Adding the second element takes 3 seconds because we need to add `{, 1, 2`. This process continues, so for the entire array set the total time is $1 + 2 + 3 + 4 + 5 + 6 + 7$.

To remedy this, Java has a special class called `StringBuilder`. It creates a string object that is mutable, so you can continue appending to the same string object instead of creating a new one each time.

Exercise 6.4.2: Rewrite the `toString()` method using `StringBuilder`.

Solution

```

public String toString() {
    StringBuilder returnSB = new StringBuilder("{}");
    for (int i = 0; i < size - 1; i += 1) {
        returnSB.append(items[i].toString());
        returnSB.append(", ");
    }
    returnSB.append(items[size - 1]);
    returnSB.append("}");
    return returnSB.toString();
}

```

Now you've successfully overridden the `toString()` method! Try printing the `ArraySet` to see the fruits of your work.

Next we will override another important object method: `equals()`

equals()

{% embed url="https://www.youtube.com/watch?v=qHuS1o97nfQ" %}

`equals()` and `==` have different behaviors in Java. `==` Checks if two objects are actually the same object in memory. Remember, pass-by-value! `==` checks if two boxes hold the same thing. For primitives, this means checking if the values are equal. For objects, this means checking if the address/pointer is equal.

Say we have this `Doge` class:

```

public class Doge {

    public int age;
    public String name;

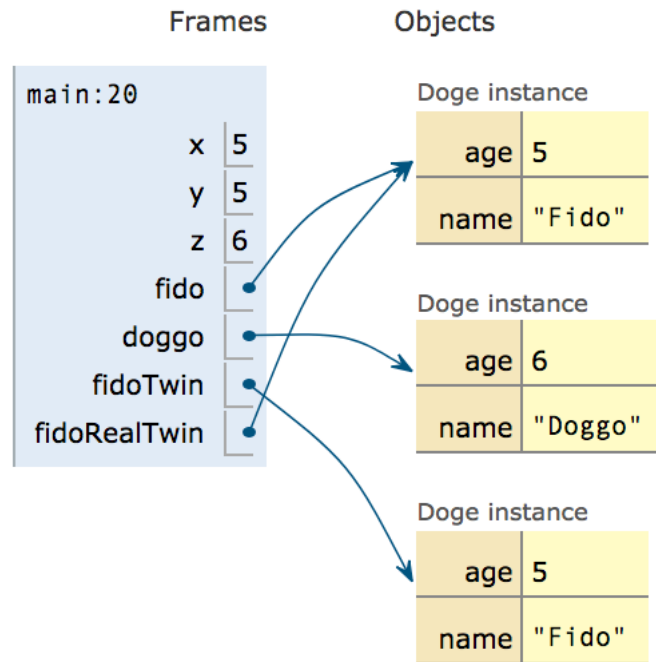
    public Doge(int age, String name){
        this.age = age;
        this.name = name;
    }
    public static void main(String[] args) {

        int x = 5;
        int y = 5;
        int z = 6;

        Doge fido = new Doge(5, "Fido");
        Doge doggo = new Doge(6, "Doggo");
        Doge fidoTwin = new Doge(5, "Fido");
        Doge fidoRealTwin = fido;
    }
}

```

If we plug this code into the java visualizer, we will see the box in pointer diagram shown below.



Exercise 6.4.3: What would java return if we ran the following?

- `x == y`
- `x == z`
- `fido == doggo`
- `fido == fidoTwin`
- `fido == fidoRealTwin`

Answers

- True
- False
- False
- False
- True

`fido` and `fidoTwin` are not considered `==` because they point to different objects. However, this is quite silly since all their attributes are the same. You can see how `==` can cause some problems in Java testing. When we write tests for our ArrayList and want to check if expected is the same as what is returned by our function, we create expected as a new arraylist. If we used `==` in our test, it would always return false. This is what `equals(Object o)` is for.

`equals(Object o)`

`equals(Object o)` is a method in the Object class that, by default, acts like `==` in that it checks if the memory address of the this is the same as o. However, we can override it to define equality in whichever way we wish! For example, for two ArrayLists to be considered equal, they just need to have the same elements in the same order.

Exercise 6.4.4: Let's write an equals method for the ArraySet class. Remember, a set is an unordered collection of unique elements. So, for two sets to be considered equal, you just need to check if they have the same elements.

Solution

```

public boolean equals(Object other) {
    if (this == other) {
        return true;
    }
    if (other == null) {
        return false;
    }
    if (other.getClass() != this.getClass()) {
        return false;
    }
    ArraySet<T> o = (ArraySet<T>) other;
    if (o.size() != this.size()) {
        return false;
    }
    for (T item : this) {
        if (!o.contains(item)) {
            return false;
        }
    }
    return true;
}

```

We added a few checks in the beginning of the method to make sure our `.equals()` can handle nulls and objects of a different class. We also optimized the function by returning true right away if the `==` methods returns true. This way, we avoid the extra work of iterating through the set.

Rules for Equals in Java: Overriding a `.equals()` method may sometimes be trickier than it seems. A couple of rules to adhere to while implementing your `.equals()` method are as follows:

1.) equals must be an equivalence relation

- **reflexive:** `x.equals(x)` is true
- **symmetric:** `x.equals(y)` if and only if `y.equals(x)`
- **transitive:** `x.equals(y)` and `y.equals(z)` implies `x.equals(z)`

2.) It must take an `Object` argument, in order to override the original `.equals()` method

3.) It must be consistent if `x.equals(y)`, then as long as `x` and `y` remain unchanged: `x` must continue to equal `y`

4.) It is never true for null `x.equals(null)` must be false

Bonus video

Create an even better `toString` method and `ArraySet.of`:

```
{% embed url="https://www.youtube.com/watch?v=tjLpeVD0KWc" %}
```

Link to the [bonus code](#)

10. Inheritance III: Iterators, Object Methods

This section covers Lists and Sets, how to throw informative exceptions, iterations, and object methods. Its contents correspond to Lecture 10.